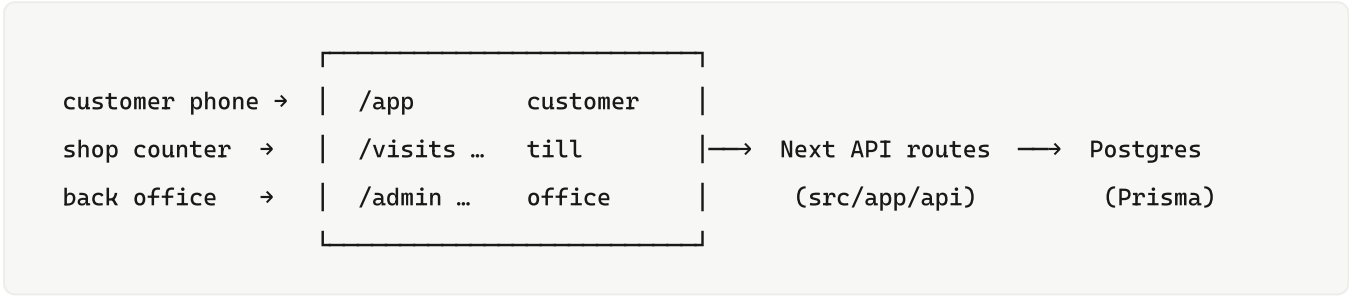
Design

How the system is put together, and why. Written to be read by whoever picks this up next — including future me.

Shape

One Next.js application, one Postgres database, three faces:



Everything shares one database and one set of rules. The rules themselves are plain functions in `src/lib`, with unit tests, and the API routes are thin wrappers over them. That is deliberate: a rule that lives in a screen gets copied and then diverges.

Module	Holds
`lib/booking.ts`	Free-slot maths, opening hours, shop-local time. Pure.
`lib/booking-server.ts`	Booking settings, hours, and finding free times in the database
`lib/booking-create.ts`	Saving an appointment — the one path both staff and customers use
`lib/payments.ts`	How a visit was paid: cash, card, InstaPay, or a mix. Pure.
`lib/finance.ts`	The money model: commission, expenses and advances by account
`lib/session-policy.ts`	How long a login lasts, per role. Pure.
`lib/roles.ts`	Who is staff, which paths belong to which door. Pure.

Roles

Role	Signs in with	Sees
`OWNER`	email	everything
`CASHIER`	email	the till, the diary, expenses, the queue
`BARBER`	email	their own day, their own customers, their own earnings
`CUSTOMER`	phone number	their own bookings and profile, nothing else

Three doors, enforced in `middleware.ts`: shop paths need a staff role, `/app` needs a customer, `/barber` needs a barber. The API enforces the same thing independently (`requireStaff`, `requireBarber`, `requireCustomer`), because middleware protects pages, not data.

A barber may mark someone arrived, started or a no-show, and nothing else: taking the money stays at the till, and they cannot move, cancel or re-price a booking. That is what keeps the diary and the takings from drifting apart.

> A path check on the prefix `"/app"` once let a customer open `/appointments`, the shop's own screen. Paths are matched segment by segment now (`isCustomerPath`), with tests — and the same care applies to `/barber` against the shop's `/barbers` screen.

The data model

The money side already existed; booking was added beside it.

```
Customer ──< Appointment >── Barber
                 │
                 ├──< AppointmentService   (name, price, length as quoted)
                 │
                 └─── Visit                (set when it is rung up)
                          │
                          ├──< VisitItem
                          └─── CashDrawerSession
```

An appointment holds a chair. A visit is money. They meet at `Appointment.visitId`, which is set once, when the customer pays. Until then the appointment carries only a quote. Ringing up is one

transaction: the visit is created and the appointment is stamped with it, so an appointment can never become two visits, and the shop can change the price at the till without rewriting what was quoted.

A customer who is served and leaves without paying is closed as `COMPLETED_UNPAID`: the chair was used, so the time stays taken, but no visit exists and no money is counted. If they come back and settle, it can still be rung up.

A walk-in is the same appointment, with `source = WALK_IN`, starting now, possibly with nobody assigned. That is what keeps one diary, one set of takings and one customer history however the customer arrived.

Decisions worth keeping:

- **Lines are snapshots.** `AppointmentService` copies the service name, price and length at booking time, as `VisitItem` already did. Raising the price of a haircut never rewrites last week's booking.
- **Cancelled frees the chair; completed does not.** One list of statuses (`BLOCKING_STATUSES`) decides this, used by the slot maths and the clash check alike.
- **"Any barber" is resolved at booking time**, not left vague: the customer gets a named barber immediately, so the shop can see who is doing what.
- **Hours are layered.** `ShopHours` is the week; `BarberWorkingHours` narrows it for one barber, never widens it. A barber with no rows simply works the shop's hours, so nothing has to be set up before booking works.
- **A barber with no `BarberService` rows does every service.** Same reason.

Time

The shop thinks in local time — "Saturday, 11:00 to 23:00" — and Egypt changes its clocks twice a year. So opening hours are stored as minutes from midnight with a weekday, and turned into real moments through the shop's timezone (`Africa/Cairo`, a setting). `shopTime()` resolves the offset twice so it stays right on the changeover day; the tests pin both a summer and a winter date.

Everything stored in the database is a UTC instant. Only the boundaries convert.

Free times

`availableSlots()` walks the working window on the shop's grid (every 15 minutes by default) and keeps a start time only if the whole service fits before closing without touching anything busy. Busy means appointments that still hold the chair, plus time off, each widened by the buffer.

The same function answers the customer app, the shop's booking modal and the reschedule screen. There is no second implementation to disagree with it.

Double booking

Free times can be stale by the time someone taps one, so the check that matters happens on save, inside a `Serializable` transaction: the clash query and the insert are one atomic step. The loser of a race gets 409 "That time has just been taken", which the screens show as a message rather than an error.

Offers

An offer is a notice, not a rule. It is shown in the customer app while it is running, and it never touches a price: the till still charges whatever the shop decides at the counter. That keeps the property everything else rests on — the takings are a record of what happened, not of what was advertised.

Money

Unchanged by the booking work, and worth restating because everything eventually lands here:

- Revenue is visit totals; tips belong to the barber and are tracked apart.
- Three accounts: **cash** (the drawer), **card** (Visa, less its commission), **InstaPay** (a bank transfer, no commission, never in the drawer). Expenses and advances come out of whichever account paid them — `outflowAccount()` decides, in one place.
- Advances are payroll, not expenses: they move money without reducing revenue.
- A tip that was not handed over in cash is paid to the barber out of the drawer, and recorded as a movement.

Reminders

A booking is worth nothing if the customer forgets it, so the app reminds them on the phone itself — a web push notification, which costs nothing per message and needs no provider, sender name or contract.

- A customer turns it on once, from their profile. The phone hands us a subscription (an address at Apple's, Google's or Mozilla's push service); we keep it against that customer, one row per phone.
- On an iPhone, notifications only exist once the app has been added to the Home Screen. The switch says so in as many words rather than failing silently.
- Two messages: **booking confirmed**, sent the moment it is booked, whoever booked it, and **a reminder**, sent the evening before. Both are written by pure functions in `reminders.ts`, in English or Arabic, in shop time.

- A nightly run (`/api/cron/reminders` , on the scheduler, behind `CRON_SECRET`) picks up everything starting within the next 20 hours that has not been reminded, sends, and stamps `reminderSentAt` . The stamp goes on even when the customer has no phone subscribed, so nobody is retried for ever.
- Sending never blocks a booking, and never fails one: it is fire-and-forget, and a subscription the push service reports as gone is deleted rather than retried nightly.

Reviews

Stars, and a line if they feel like writing one. The rules are deliberately narrow, because a rating nobody believes is worse than no rating at all:

- Only a finished appointment can carry a review — including one they left without paying, since they still sat in the chair. Cancelled and missed appointments are silent.
- One per appointment. Writing it again is a change of mind, not a second opinion: the row is updated.
- The window closes after a month. Nobody is asked in March what they thought of a haircut in January.
- The evening run asks yesterday's customers once, on the phone. Ignoring the ask is an answer too, so it is never repeated.

The shop's only power over a review is `hiddenAt` : the owner can take an abusive one off the app, where it counts for nothing and is shown to nobody, but it is not deleted and the activity log records who hid it. A shop that can quietly delete criticism has a rating worth nothing, including to itself.

Averages are `null` when nobody has said anything — never zero, which reads as "terrible" rather than "new".

Loyalty

The paper punch card without the paper. A stamp for every visit the customer paid for; after however many the shop sets, a reward is waiting.

Two things it deliberately does not do:

- It never changes a price. A full card is a note on the till screen while the customer stands there; the cashier decides what to charge and ticks "reward given", which records it. The takings stay a record of what was taken, which is the same reason offers are notices rather than rules.
- It never counts anything but a paid visit — a `Visit` row. A booking, a no-show, or someone who left without paying earns nothing.

Rewards earned are `floor(visits / required)`; rewards given are rows in `LoyaltyRedemption`; what is waiting is the difference, never below zero — a reward handed over early is the shop's kindness, not the customer's debt.

Insights

A second reading of the same rows the money is made of: when the shop is busy, who did the work, how many faces were new, and whether people come back. Nothing here has its own store of numbers, so an insight can never disagree with the day journal.

Two of the questions cannot be answered from a window, and are not:

- **New versus returning** compares each visit against that customer's first visit ever. Counted inside the window instead, everybody would look new the moment the window opened.
- **Retention** reads each customer's whole history: how many came back at all, the typical gap between visits, and which regulars have not been in for two months — the last of which is the shop's best call list.

Times are bucketed in shop time, so a visit at half past midnight belongs to the night the shop worked.

Customer groups

Six groups, worked out from the visits every time they are asked for: newcomers, regulars, slipping away, lapsed, big spenders, and the ones who tried the shop once. Nothing is stored — a customer joins a group by living their life and leaves it the same way, so a group can never go stale behind the shop's back.

The thresholds are written down in one place (`segments.ts`) because a shop that cannot say exactly what "a regular" means will argue with the number on the screen. Slipping and lapsed are kept apart deliberately, so the same person is not chased twice.

The owner can send one group a notification. Three limits, because the quickest way to lose someone's notifications for ever is to send them something they did not ask for:

- the owner alone, never the cashier and certainly never a customer;
- the same group at most once every twelve hours;
- only customers who turned notifications on — and the screen says how many that is before the message is written, which is usually far fewer than the group.

Every send is written to the activity log with the words that were sent. For everyone else, the screen copies the phone numbers so the shop can ring them.

Telling a friend

Every customer has a six-character code in their app, made the first time they look for it. A new customer types it when they sign up, and the shop owes a reward once that friend has **paid for a haircut** — not when they sign up, because an account costs nothing to make.

The code leaves out the characters people confuse when reading one out across a counter: no 0/O, no 1/I/L, no 5/S. A code nobody recognises is ignored rather than refused — the account matters more than the referral.

Nothing about whether a referral is earned is stored: it is read from the visits, so it cannot drift. Only the reward the shop handed over is written down, along with who gave it.

Birthdays

Only the day and the month are ever used — the year a customer gave is their business, and the shop has no reason to work out anybody's age. The evening run wishes a happy birthday once a year, in shop time, and stamps the customer whether or not a phone was listening so the greeting is a once-a-year thing rather than a nightly attempt. Somebody born on 29 February is greeted on the 28th in a common year rather than three years out of four.

The shop sees the birthdays coming up on the customer groups screen, because most customers have no app and a phone call is what actually happens.

Brass & Ink

The apps are dressed from one set of tokens at the top of `globals.css`: a ground, four strengths of ink, a hairline, and brass. Nothing in the customer or barber app names a colour of its own, which is what makes the whole thing turn from night to day by swapping that block — and what stops a screen built next month drifting away from the rest.

- **Dark is the default**, because that is when people book and what the room looks like. A switch in the header turns it over, and the choice is remembered on that phone. The theme is settled before the first paint, so nobody sees the wrong one flash past.
- **Numbers are set in Bodoni** — times, prices, counts — which is what makes an appointment read like a price on a menu. Everything else is Manrope, and Arabic stays in Cairo.
- **Square corners and hairline rules.** Three pixels of radius, one brass line along the top of a lifted card, and nothing else decorative.
- The shop's own till screens are deliberately *not* on these tokens. They are worked on for eight hours a day and stay white.

The usual

A regular books the same barber, the same cut, at the same sort of hour, and makes the same four taps every time. Their own history already says all of it, so the app offers it back: the cut, the barber, and the three free times nearest the hour they normally come.

It is read from appointments rather than visits, because an appointment knows what was asked for and who was asked for. Only ones they sat through count — a cancelled booking says nothing about a habit — and it takes two before the app will call anything usual, so it never guesses from a single visit.

Two things it will not do. It will not offer a barber who has left or been taken out of the app, or a service the shop has stopped doing: the cut survives, the barber quietly drops to "any barber", and if the service itself has gone there is no usual to offer. And it will not book anything by itself — a booking holds a chair, so the customer always presses the button.

The owner's nightly line

One notification at closing: what the shop took and how it was paid, how many people came, what the no-shows cost, and how many bookings stand for tomorrow. It answers the question the owner would otherwise open a laptop for.

Counted from the same rows as the day journal, over the same shop day — its own midnight through to 3am — so the line and the reports can never disagree. It goes to owners who have turned notifications on, on whichever device they turned them on: subscribing is the opt-in, so there is no setting to forget to switch on. A day with no visits, no no-shows and nothing booked for tomorrow sends nothing at all, because a closed day is not news.

"It has been a while"

Every regular has a rhythm — three weeks, five weeks — and the insight screens already work it out. This uses each customer's own rhythm to notice when they are late, and says so once, with their barber's name and their own number of weeks. The tap lands on the home screen, where **the usual** is waiting with times, so a nudge turns into a booking in two taps.

The whole design problem is restraint. A shop that writes to people who did not ask loses them, and loses their notifications with them, so the rule refuses far more often than it sends:

- three visits before the shop presumes to know anyone's rhythm;
- a week of grace past their own gap before they count as late;
- never while they already have an appointment booked;

- never twice within three months;
- and nothing at all once they are two months beyond due — somebody that far gone has left rather than forgotten, which is a different conversation and belongs to the customer groups screen.

Twenty-five a night at most, longest overdue first, so a shop turning it on for the first time does not message three hundred people at once. Off until the owner asks for it, on the booking settings screen.

Selling a product

The shop has always bought stock and used it on the floor. What it could not do was **sell** a bottle of beard oil and have that show up anywhere — so wax and shampoo sold over the counter were money leaving the shop untracked.

A visit now carries product lines beside its service lines, kept in their own table on purpose: a service line feeds a barber's commission and the length of an appointment, and a bottle of oil feeds neither. Both land on the visit total, which is what the drawer, the day journal and every report already read, so a product sale needs no new money path and cannot disagree with one.

The rules around it:

- A product is for sale only when it is active **and** somebody has priced it. Zero means "used on the floor", which is most of the shelf.
- The till cannot sell what the branch does not hold — two lines of the same thing are added together before the shelf is checked. A shop that can sell stock it does not have ends up with a figure nobody believes, and then nobody counts.
- The money and the stock move in the same transaction. Undoing the visit puts the bottle back, and the activity log remembers what was on it.
- The percentage discount comes off the services only. A product is sold at the price the cashier typed, so taking a further percentage off it would double-count the kindness.
- Name and cost are copied onto the line at the moment of sale, so a later price change never rewrites what happened — the same reason a service line copies its price in.

The period report breaks the counter out of the takings: what products brought in, what that stock cost, and how many left the shelf. A bottle and a haircut are two different businesses with two different margins.

Sessions

Shop devices sit on a counter, so a staff login ends after 2 hours, or 30 minutes idle. A customer's phone is their own, so a customer login lasts as long as they keep using it (30 days idle). One policy

module, `role` as the input; the token's own expiry carries it, so the middleware enforces it too.

Conventions

- Screens are client components; the API is the security boundary.
- Every route checks its own permission. None trusts the middleware.
- Anything that rewrites or removes money writes an `AuditLog` line, fire-and-forget.
- Both languages, always: a new string is added to `messages/en.ts` and `messages/ar.ts` together, and the type makes a missing Arabic key a compile error.
- New rules arrive as a tested function in `lib/` first, then a screen.
